Digital Image Processing Lab (NCSH302) **Session: Monsoon 2026-27**
B. Tech-5th Sem. (CSE-Hons) **Date: 27.10.2026**

Assignment-10 (Image Segmentation - III and Image Compression - I)

---

1. Read the grayscale image $messi.png$ and implement the *Canny Edge Detection algorithm* using the following steps: (I) smooth the image using a 5×5 Gaussian filter with $\sigma = 1$, (II) compute the gradient magnitude $G = \sqrt{G_x^2 + G_y^2}$ and direction $\theta = tan^{-1}\left(\frac{G_y}{G_x}\right)$ using Sobel operators, taking $\theta = 90^\circ$ when $G_x = 0 \, and \, G_y \neq 0$, (III) apply non-maximum suppression by normalizing the gradient direction to $[0^\circ, 180^\circ)$ and selecting the nearest orientation among $0^\circ, 45^\circ, 90^\circ, 135^\circ$, and (IV) perform double thresholding and edge linking using hysteresis with lower threshold $T_L = 100$ and upper threshold $T_H = 200$, retaining weak edge pixels connected to strong edge pixels through 8-connectivity. Display the original grayscale image, Gaussian-smoothed image, gradient magnitude image, non-maximum suppressed image, and final Canny edge image.

[**10 Marks**]

Sample Input:



Sample Output:

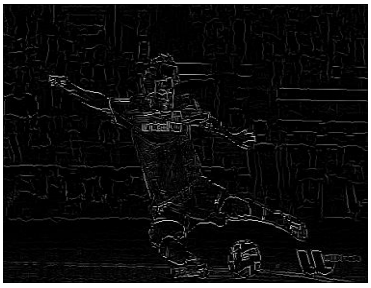
Gaussian smoothed image



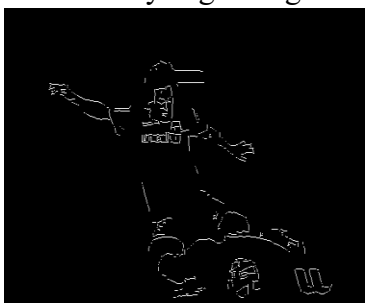
Gradient magnitude (display clipped to 0 - 255)



Non-maximum suppression (display clipped to 0 - 255)



Final Canny edge image

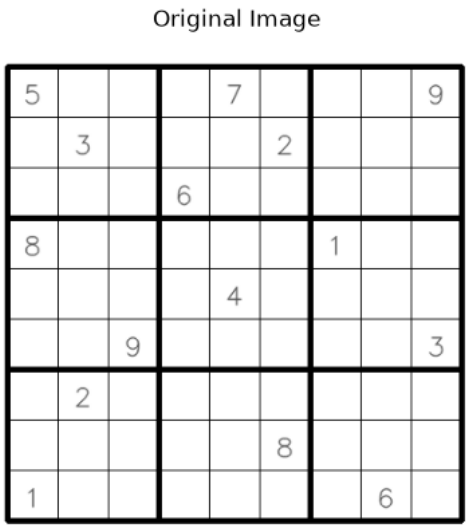


2. The Hough Transform is a technique used to detect straight lines in an image, even when some portions of the lines are broken. Consider the image $sudoku.png$ and perform the following tasks using Python.
    1. Read the image $sudoku.png$ and convert it into a grayscale image.
    2. Apply the Canny edge detection algorithm with lower and upper thresholds of $50\ and\ 150$, respectively.
    3. Initialize an accumulator matrix using the parameters ρ and θ, where ρ represents the perpendicular distance from the origin and θ represents the angle of the line's normal.
    4. For each detected edge pixel (x, y), calculate $\rho = xcos\theta + ysin\theta$ for $\theta = 0^{\circ}, 1^{\circ}, \dots, 179^{\circ}$.
    5. Increment the corresponding accumulator cell by one vote for each calculated (ρ, θ) pair.
    6. Identify the accumulator cells having at least 100 votes and use their corresponding (ρ, θ) values to determine the straight lines.
    7. Draw the detected lines in red on the original image.

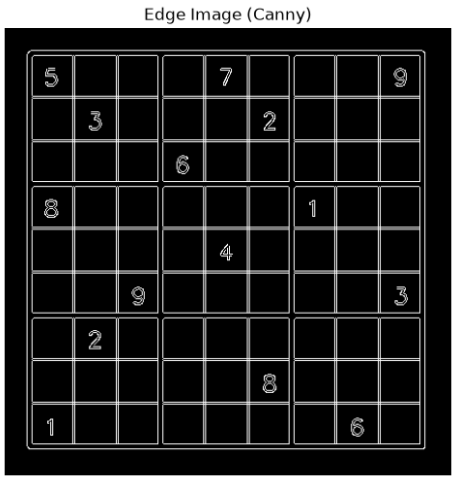
    Display the edge image, and final image with detected lines in separate plots.

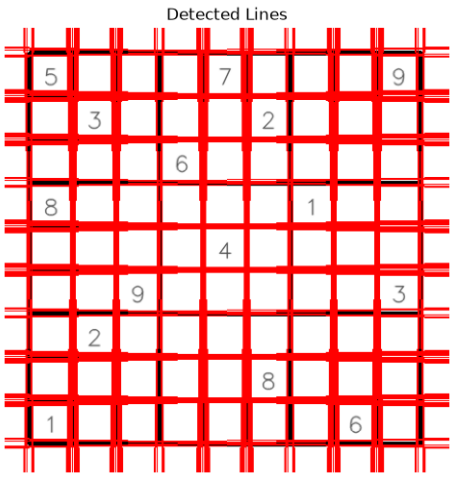
[10 Marks]

Sample Input:



Sample Output:





3. Read the grayscale image $cameraman.tif$ and resize it to 256×256. Convert the image into binary form using threshold $T = 128$, where $B(x,y) = 1\ if\ I(x,y) \geq T$, otherwise $B(x,y) = 0$. Implement horizontal and vertical Run-Length Coding (RLC) independently, encoding each row or column as alternating runs beginning with zeros and representing each run length using 9 $bits$. Decode both compressed representations to reconstruct the binary image. Compute the compression ratio $CR = \frac{S_o}{S_c}$ and bits per pixel $bpp = \frac{S_c}{MN}$, where $S_o = MN$ bits and $S_c = 9R$ bits, with R denoting the total number of encoded runs. Display the original grayscale image, binary image, and both reconstructed images, and compute the specified metrics for horizontal and vertical RLC.

[10 Marks]

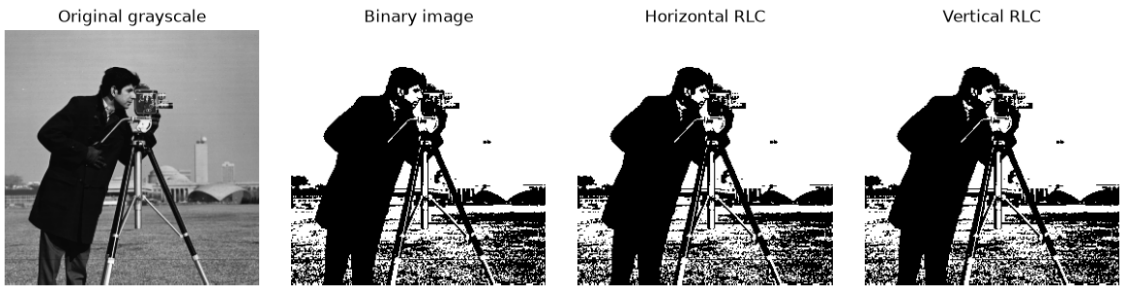
Sample Input:



Sample Output:

HORIZONTAL RLC
Total runs: 6247
Compressed bits: 56223
Compression ratio: 1.1656
Bits per pixel: 0.8579
Exact reconstruction: True

VERTICAL RLC
Total runs: 6629
Compressed bits: 59661
Compression ratio: 1.0985
Bits per pixel: 0.9104
Exact reconstruction: True



4. Read the grayscale image $cameraman.tif$ and resize it to 256×256. Quantize the image into $16\,gray$ levels using $Q(x,y) = \left\lfloor \frac{I(x,\ y)}{16} \right\rfloor$. Calculate the occurrence probability $p_{n_i} = \frac{n_i}{MN}$ of each gray level and implement Shannon - Fano Coding by sorting symbols in descending probability order and recursively dividing them into two consecutive groups with the minimum difference in total probability, assigning binary digits 0 and 1 to the first and second groups, respectively. Resolve equal-probability ties by ascending gray level and partition ties by choosing the smaller first group. Encode and decode the quantized image, then compute the compression ratio $CR = \frac{S_o}{S_c}$ and coding efficiency $\eta = \frac{H}{L_{avg}} \times 100\,\%$, where $S_o = 4MN$ bits, $S_c = \Sigma\, n_i l_i$, $H = -\sum p_i \log_2 p_i$, and $L_{avg} = \sum p_i l_i$, with l_i representing the codeword length. Display the original grayscale image, quantized image, reconstructed image, and Shannon - Fano code table and compute the specified metrics.

[10 Marks]

Sample Input:



Sample Output:

Gray level	Frequency	Probability	Codeword
10	14770	0.225372	00
0	10346	0.157867	010
9	9092	0.138733	011
11	7783	0.118759	100
8	6241	0.095230	101
7	5241	0.079971	1100
1	3186	0.048615	1101
6	3100	0.047302	11100
2	1388	0.021179	11101
5	1179	0.017990	111100
3	1112	0.016968	111101
4	925	0.014114	111110
12	562	0.008575	1111110
13	255	0.003891	11111110
14	242	0.003693	111111110
15	114	0.001740	111111111

SHANNON-FANO RESULTS

Entropy: 3.2445 bits/symbol
Average code length: 3.2737 bits/symbol
Compressed bits: 214548
Compression ratio: 1.2218
Coding efficiency: 99.11 %
Exact reconstruction: True

